Founder Technical Demo Briefing

Prepared for a 60 to 75 minute founder call about what is built, what is not built, how the system is configured, how data moves through thun.ai, how AI works inside the product, and how strong the product is today.

1. The One-Minute Executive Answer

Use this when the founder opens with: “Where are we?”

thun.ai is no longer just an idea or a slide prototype. We have a working multi-layer product architecture across mobile, backend, edge, and AI runtime code. The strongest completed parts are the mobile journey, route stress scoring, stress-index computation, intervention logic, post-drive feedback, offline sync, backend security middleware, privacy APIs, and test-covered backend behavior. The honest gap is hardware validation: the RV1126 edge path is designed and has guardrails, but real in-vehicle CAN, camera, BLE, RKNN model execution, and road-condition validation are still the next milestone. So the product is strong for a founder demo and pilot planning, but not yet ready to claim OEM production safety performance.

Shorter version:

We are demo-strong, architecture-strong, and backend/mobile-test-strong. We are not yet hardware-field-validated.

2. Recommended Call Structure

This agenda fills more than an hour without sounding padded.

Time	Section	Purpose
0-5 min	Strategic framing	Explain why thun.ai exists and why it is not ADAS/autonomy.
5-15 min	Product walkthrough	Show onboarding, pre-drive route scoring, drive HUD, confidence corridor, post-drive report, therapist.
15-30 min	System architecture	Explain mobile, edge, backend, Python AI runtime, external providers, database.
30-42 min	Data routes and communication	Explain how OBD, watch, camera, route data, LLM, sync, and privacy flows connect.
42-52 min	Resilience and safety design	Explain speed gate, cooldowns, fallbacks, offline queue, audit, rate limits, startup validation.
52-62 min	Done vs not done	Be very clear on what is implemented, partial, and roadmap.
62-70 min	Hardware bring-up and testing timeline	Explain what happens when hardware is configured and how long validation takes.

70-75 min	Closing ask	Align on pilot, hardware, team, funding, or founder decision.

3. Positioning To Open With

Do not begin with model names or frameworks. Begin with the safety thesis.

thun.ai is a driver-state-aware safety layer for anxious and under-confident drivers. We are not trying to replace the driver. We are trying to prevent the driver from getting overwhelmed in the exact moments where panic, hesitation, freezing, or poor judgment increases risk.

The sharp distinction:

- ADAS asks: What is happening outside the car?
- thun.ai also asks: What is happening inside the driver?
- Navigation asks: What is the fastest route?
- thun.ai asks: What is the calmest safe route for this specific driver?
- A normal alert system says: Danger.
- thun.ai says: Here is the calm, minimal next action.

4. What Is Done Today

Mobile Product Surface

Built in the React Native app:

- Onboarding profile capture for driver sensitivity, triggers, language, and vehicle width.
- Home/dashboard and drive history.
- Pre-drive route scoring and route comparison.
- Trigger preferences such as narrow lanes, U-turns, highway merges, and flyovers.
- In-drive HUD, stress gauge, interventions, breathing cue, lane guidance, emergency override, and stall protocol.
- Confidence corridor for tight passages, including predicted spare width, left/right clearance, recommended speed, stop/caution/clear states, and confidence memory.
- Post-drive reporting flow.
- Stationary-only therapist screen.
- Settings for profile, token provisioning, privacy, and runtime status.
- Local SQLite storage with 90-day retention.
- Offline sync queue and replay.
- Secure key manager with Android keystore-backed database key path for production and a development fallback.

Code evidence:

- mobile/src/screens/OnboardingScreen.js
- mobile/src/screens/PreDriveScreen.js
- mobile/src/screens/DriveScreen.js
- mobile/src/screens/PostDriveScreen.js
- mobile/src/screens/TherapistScreen.js
- mobile/src/services/StressIndexService.js
- mobile/src/services/IVISEngine.js
- mobile/src/services/ConfidenceCorridorService.js
- mobile/src/services/SyncService.js
- mobile/src/services/SecureKeyManager.js

Backend Product Surface

Built in the Node.js/Express backend:

- Firebase token verification and backend JWT issuance.
- Protected drive session CRUD.
- Route accident-zone scoring endpoint.
- Post-drive confidence narrative generation.
- Synthetic practice scenario generation.
- Therapist chat proxy.
- Confidence trajectory endpoint.
- Privacy consent, data export request, and deletion request APIs.
- PostgreSQL schema and Flyway migrations.
- Request ID tracing.
- Per-user Redis-backed rate limits with fail-open fallback.
- LLM-specific and therapist-specific rate limits.
- Audit logging for user-impacting actions.
- Sanitized error responses with request IDs.
- Startup readiness validation.
- Health endpoints.
- Optional Sentry/error tracking wrapper.

Code evidence:

- backend/src/index.js
- backend/src/routes/auth.js
- backend/src/routes/drive.js
- backend/src/routes/route.js
- backend/src/routes/feedback.js
- backend/src/routes/privacy.js
- backend/src/services/llmService.js
- backend/src/middleware/rateLimiter.js
- backend/src/middleware/audit.js
- backend/migrations/V1__initial_schema.sql
- backend/migrations/V2__add_audit_table.sql
- backend/migrations/V3__add_privacy_controls.sql

Edge Runtime Surface

Built in the C++ edge layer:

- RV1126-oriented C++ IVIS engine.
- OBD parser.
- C++ stress index implementation.
- Intervention dispatcher.
- Runtime configuration from environment variables.
- Edge self-test for CAN interface, camera device, BLE key file, YOLO model, and lane model.
- BLE payload encryption path using AES-256-GCM when OpenSSL is available.
- Latency instrumentation inside the tick loop.

Code evidence:

- edge/src/ivis_engine.cpp
- edge/src/runtime_config.h
- edge/src/ivis_selftest.cpp
- edge/src/stress_index.cpp
- edge/src/intervention_dispatcher.cpp

- edge/src/crypto_utils.h

Important nuance:

The edge hardware skeleton is serious, but production CV inference is not complete. The current runCVInference() path still returns stubbed/simulated CV signals, even though the hardware initialization path loads model files under the real-hardware build flag. Do not claim production-grade emergency vehicle, lane, or side-clearance detection until real RKNN inference is wired and tested on the target device.

Python AI Runtime

Built in the Python core:

- YAML plus environment-based configuration.
- Provider factories for LLM, SLM, VLM, voice, and perception.
- Main ThunaiEngine orchestrator.
- Pre-drive, IVIS, therapist, post-drive modules.
- Hardware readiness monitor with 2-second disconnect rule and 50 ms latency budget.
- Circuit breaker utility for external API calls.
- Synthetic dataset export for downstream model training.
- Pilot and production config profiles.

Code evidence:

- src/thunai/engine.py
- src/thunai/config.py
- src/thunai/hardware.py
- src/thunai/circuit_breaker.py
- src/thunai/features/ivis.py
- src/thunai/features/post_drive.py
- src/thunai/features/therapist.py
- config/default.yaml
- config/profiles/pilot.yaml
- config/profiles/production.yaml

5. What Is Not Done Yet

Say this plainly. A founder usually wants confidence, but they also want judgment.

Hardware and Sensor Validation

Not yet fully done:

- Real RV1126 board validation with actual CAN input.
- Real camera initialization and sustained frame-rate testing on target hardware.
- Real RKNN inference for emergency vehicle, lane, driver state, and side-clearance models.
- Real BLE interoperability with phone/watch pairing.
- Real in-vehicle latency measurements under sensor load.
- Controlled road tests with repeatable scenarios.

Production Integration Gaps

Not yet production-ready:

- Google Maps route calls should be backend-proxied before production. Some route logic still calls Google directly from mobile.
- Sarvam TTS is callable from mobile; for production, key handling and proxy strategy need tightening.
- TLS certificate pins in mobile are placeholders.
- Redis fallback is good for development, but production needs real Redis.
- Circuit breaker state is in memory in backend; production multi-instance setups should move breaker state to Redis or another shared store.
- Full production sign-in UX is not finished; pilot token provisioning exists.
- Consent APIs exist, but a full production-grade consent UX and deletion job pipeline still need completion.
- Monitoring, alerting, load testing, and CI/CD migration automation need to be formalized.

Product Intelligence Gaps

Partially done, not final:

- Route scoring exists, but heavy vehicle and narrow-lane scoring are still heuristic.
- Confidence corridor exists, but current live clearance can be route-model-driven unless real leftClearanceCm and rightClearanceCm CV signals are provided.
- Emergency override exists in logic, but real emergency vehicle CV detection is not field-validated.
- Lane guidance exists, but directional lane correction is heuristic until lane model integration is complete.
- Personal threshold calibration needs pilot data.

6. Product Strength Assessment

Use this framing if the founder asks: “How strong are we with the product?”

Area	Strength Today	Reason
Founder demo	Strong	Full user journey exists: onboarding, route, drive, corridor, post-drive, therapist.
Mobile product logic	Strong for prototype/pilot	Core services are implemented and mostly test-covered.
Backend API	Strong for pilot foundation	Auth, validation, rate limits, audit, privacy, LLM fallback, migrations, and tests exist.
AI architecture	Medium-strong	Clear separation of deterministic safety logic vs generative coaching; fallback paths exist.
Offline/degraded behavior	Medium-strong	Local storage, sync queue, OBD/watch reconnect, and fallback responses exist.
Edge architecture	Medium	C++ modules and self-test exist, but real CV/vehicle hardware validation is pending.
OEM production readiness	Not yet	Hardware validation, CV integration, security hardening, monitoring, load testing, and compliance workflow remain.

Suggested numeric language:

- Demo readiness: 75 to 85 percent.
- Pilot software readiness: 60 to 70 percent.
- Hardware field readiness: 35 to 45 percent until first RV1126 validation is complete.
- OEM production readiness: 45 to 55 percent, depending on how strict the production definition is.

Avoid saying “we are production ready” unless you qualify exactly what layer you mean.

Better phrasing:

The product is strong enough to demonstrate the category and run a controlled pilot plan. It is not yet strong enough to make safety-certified OEM production claims.

7. Hardware Configuration

Target Edge Configuration

The target edge unit is:

- SoC: Rockchip RV1126.
- CPU: ARM Cortex-A7 around 1.5 GHz.
- NPU: RKNN NPU, around 2 TOPS class.
- OS: Rockchip Linux/BSP path.
- Camera: GMSL2 camera feed into MIPI CSI, configured in code for 640 x 480 NV12 at 30 fps.

- Vehicle telemetry: CAN bus, ISO 15765-4, 500 kbps target.
- Wireless: BLE 5.0 for edge-to-mobile notifications.
- Security: BLE secure pairing plus encrypted stress payload path using AES-256-GCM when OpenSSL is available.

Required Edge Environment

The edge self-test and real-hardware init expect:

```
IVIS_CAN_IFACE=can0
IVIS_CAMERA_DEVICE=/dev/video0
IVIS_BLE_DEVICE_NAME=IVIS-Edge
IVIS_BLE_KEY_FILE=/opt/ivis/secrets/ble.key
IVIS_YOLO_MODEL_PATH=/opt/ivis/models/yolo_emergency.rknn
IVIS_LANE_MODEL_PATH=/opt/ivis/models/lanenet.rknn
```

Edge Bring-Up Sequence

The ideal device sequence is:

1. Flash/load Linux BSP and confirm shell access.
2. Install or copy the IVIS binary/library.
3. Confirm CAN interface appears under /sys/class/net/can0.
4. Confirm camera node appears at /dev/video0.
5. Place BLE key file under /opt/ivis/secrets/ble.key.
6. Place RKNN model files under /opt/ivis/models/.
7. Run ivis_selftest.
8. Start the IVIS service.
9. Pair mobile app to the edge BLE device.
10. Start controlled sensor feed testing.

How Long Hardware Testing Takes Once Configured

If the hardware and models are already physically available:

Test Stage	Time Estimate	Meaning
Pre-flight file/config check	30 to 60 min	Env vars, model paths, BLE key, app/backend token.
Edge self-test	15 to 30 min	Confirms CAN path, camera node, BLE key, model files.

Bench smoke test	2 to 4 hours	Runs simulated or static sensor feed, checks logs and BLE output.
Static vehicle test	4 to 8 hours	Connects to parked vehicle, verifies CAN/OBD, camera, BLE, mobile display.
Low-speed controlled drive	1 to 2 days	Closed route, low-risk scenarios, intervention timing, disconnect behavior.
Repeatable pilot readiness test	3 to 5 days	Multiple routes, day/night, traffic density, route stress scoring, sync recovery.
Credible first pilot sign-off	1 to 2 weeks	Enough repeated evidence to let real users try it under supervision.

Founder-ready wording:

If the hardware is already correctly configured, we can know the unit is alive the same day. But proving it is reliable in a moving vehicle takes days, and proving it is pilot-safe takes one to two weeks of controlled repeat testing.

8. Software Configuration

Mobile

- Framework: React Native 0.73.
- Main data store: local SQLite.
- Local retention: 90 days for drive history.
- Secure storage: Android keystore bridge for database key in production; dev fallback in development.
- Network: Axios, 10 second timeout.
- OBD: Bluetooth Classic, ELM327-style PID polling.
- Watch: BLE heart-rate service, HR/HRV parsing.
- Voice: Sarvam AI when configured, native TTS fallback.
- Offline: Sync queue stored locally and replayed on reconnection or app foreground.

Backend

- Runtime: Node.js 18+.
- Framework: Express 4.
- Database: PostgreSQL.
- Migrations: Flyway.
- Auth: Firebase ID token verification → backend JWT, default 7 days.
- Rate limits: 100 requests/min global per user/IP, 10 requests/min LLM, 5 therapist conversations/day.
- External AI providers: Gemini, Claude, OpenAI fallback path in backend code.

- Monitoring/error path: Sentry wrapper available when DSN is configured.

Python Runtime

- Runtime: Python 3.10+.
- Config: config/default.yaml, config/profiles/pilot.yaml, config/profiles/production.yaml, environment overrides.
- Provider layers:
    - LLM: Gemini, OpenAI, stub.
    - SLM: Ollama, Phi-3, Mistral, stub.
    - VLM: Gemini, Ollama/LLaVA, stub.
    - Voice: Sarvam, ElevenLabs, system, stub.
    - Perception: YOLO, MobileNet, stub.

Production Startup Requirements

Backend requires:

- DATABASE_URL
- Strong JWT_SECRET
- Firebase credentials
- ALLOWED_ORIGINS in strict/prod mode
- SENTRY_DSN in strict/prod mode
- At least one LLM provider API key for real feedback

Python production profile blocks stub providers for SLM, perception, LLM, voice, and navigation.

9. How Data Moves Through The System

This is the section to use when the founder asks: “How did we connect different sources via different routes?”

Route 1: Onboarding and Profile

Flow:

1. User enters driver profile, triggers, language, threshold sensitivity, and vehicle dimensions.
2. Mobile saves profile locally.
3. Backend profile update can store anxiety profile when authenticated.

4. Profile informs route scoring, stress threshold, TTS language, and confidence corridor width.

Why it matters:

The system is personalized from day one. The same traffic condition can be acceptable for one driver and overwhelming for another.

Route 2: Pre-Drive Route Scoring

Flow:

1. Mobile asks Google Maps for candidate routes.
2. Mobile scores live traffic from duration vs duration-in-traffic.
3. Mobile estimates highway merges from route step instructions.
4. Mobile calls backend /route/accident-zones with route polyline.
5. Backend scores polyline against accident zones in PostgreSQL/PostGIS style query path.
6. Mobile estimates heavy vehicle exposure and narrow-lane exposure.
7. Mobile applies user trigger preferences.
8. Routes are sorted by lowest anxiety score.

Factors:

- Heavy vehicle density: 25 percent.
- Highway merge frequency: 20 percent.
- Accident zones: 25 percent.
- Narrow lanes: 15 percent.
- Live traffic: 15 percent.
- Custom trigger penalties are added on top.

Current gap:

Heavy vehicle and narrow-lane detection are heuristic. For production, these should become backend-backed city data or map enrichment.

Route 3: Real-Time Drive Tick

Flow:

1. OBDService polls vehicle telemetry every 200 ms.
2. WatchService streams HR/HRV from BLE heart-rate service.
3. Edge or CV path supplies tailgating risk, lane drift, head pose, emergency vehicle, and later clearance.

4. StressIndexService computes composite stress score from available inputs.
5. IVISEngine decides whether an intervention should fire.
6. TTSService speaks only if under the speed gate.
7. HUD and breathing cues update the DriveScreen.
8. LocalStorage logs interventions and session data.

Stress index weighting:

- OBD/driving signals: 40 percent.
- Biometrics: 40 percent.
- CV signals: 20 percent.

OBD sub-factors:

- Speed variance.
- Harsh braking.
- Harsh acceleration.
- Gear mismatch.

Biometric sub-factors:

- Heart rate above baseline.
- HRV below baseline.

CV sub-factors:

- Tailgating risk.
- Lane drift.
- Head pose/distraction score.

Key safety line:

The LLM is not deciding whether the car is safe. The real-time safety path is deterministic: sensors → stress score → thresholds → rules → speed-gated intervention.

Route 4: Confidence Corridor

Flow:

1. Route scoring identifies possible narrow-lane/tight-space exposure.
2. ConfidenceCorridorService creates a preview using vehicle mirror width, narrow-lane score, heavy vehicle score, anxiety score, and driver preferences.
3. During the drive, corridor state can use real CV leftClearanceCm and rightClearanceCm if available.

4. If real clearance is not available, the current demo can simulate a route-model corridor.
5. The driver sees clear/caution/stop guidance, spare centimeters, left/right clearance, and recommended speed.
6. Post-drive summary records whether passages were successful, cautious, or blocked.
7. Confidence memory is updated.

Founder wording:

This is one of the strongest product ideas because it turns a vague fear into a measurable decision: will I fit, how much room do I have, and should I continue or stop?

Honest gap:

The visual/product logic is implemented, but production-grade side-clearance sensing still needs real CV or sensor-fusion validation.

Route 5: Post-Drive Feedback

Flow:

1. Mobile ends drive and compiles stress events, route metadata, telemetry summary, and confidence corridor outcomes.
2. Mobile sends feedback request to backend through protected API.
3. Backend rate-limits the LLM endpoint.
4. Backend checks whether a narrative already exists for the session.
5. Backend calls LLM providers with request ID tracing.
6. Backend stores narrative and confidence trajectory atomically.
7. Mobile displays confidence report and scenarios.

Backend LLM fallback:

1. Gemini if configured.
2. Claude if Gemini fails or times out.
3. OpenAI if Claude fails.
4. Deterministic fallback narrative if all providers fail.

Controls:

- Global timeout: 30 seconds.
- Per-provider timeout: 8 seconds.
- Circuit breaker: opens after 5 consecutive provider failures for 5 minutes.
- Cached narratives avoid repeated LLM calls.

Route 6: Therapist

Flow:

1. User opens therapist only when stationary.
2. Mobile/engine checks stationary condition by RPM/speed path.
3. Backend therapist endpoint receives last conversation messages and context.
4. Backend sanitizes messages and limits history.
5. LLM generates short, calm, CBT-aligned response.
6. TTS speaks response if safe.

Important safety claim:

Therapist is not a medical product and not active during driving. It is a parked recovery and coaching feature.

Route 7: Sync, Privacy, and Offline

Flow:

1. Mobile stores profile, sessions, interventions, privacy settings, and sync queue locally.
2. If backend is offline, retryable requests are queued.
3. When network returns or app foregrounds, SyncService flushes pending requests.
4. AuthSessionService attaches backend JWT when provisioned.
5. PrivacyService can sync consent, export requests, and deletion requests.

Why it matters:

Poor connectivity should not break the drive. The safety loop runs locally, and cloud-dependent features replay later.

10. Communication Strength And Resilience

Communication Links

Link	Protocol	Purpose	Strength Today
OBD adapter → mobile	Bluetooth Classic	Speed, RPM, throttle, engine load, coolant, inferred gear	Implemented with reconnect/backoff logic. Needs real dongle compatibility matrix.
Watch → mobile	BLE heart-rate service	HR and HRV	Implemented with reconnect/backoff logic. Needs device compatibility testing.
Edge → mobile	BLE	Stress/intervention payloads	C++ path and encryption design exist. Needs

			target hardware validation.
Mobile → backend	HTTPS/TLS	Auth, drive sessions, route scoring, feedback, privacy	Implemented. TLS pin placeholders still need real pins.
Backend → PostgreSQL	pg pool	Users, sessions, audit, privacy, confidence	Implemented. Needs production DB provisioning and migrations in CI/CD.
Backend → LLM providers	HTTPS SDK/API	Feedback, therapist, scenarios	Implemented with timeout, fallback, and circuit breaker.
Mobile → Google Maps	HTTPS	Candidate routes	Implemented but should be proxied before production.
Mobile → Sarvam/native TTS	HTTPS/native	Voice output	Implemented with fallback. Production key handling needs tightening.

Resilience Built Today

- Sensor fallback in stress scoring: if one source is missing, available weights are normalized instead of returning unusable output.
- OBD reconnect with exponential backoff up to 30 seconds.
- Watch reconnect with exponential backoff up to 30 seconds.
- TTS fallback from Sarvam to native TTS.
- Voice speed gate above 60 km/h in mobile.
- Intervention cooldown of 30 seconds.
- Emergency override priority path.
- Stall protocol when RPM is 0, speed is 0, and stress is high.
- Offline request queue and replay.
- Local 90-day drive history.
- Backend request ID tracing.
- Sanitized backend error responses.
- Per-user rate limiting.
- LLM timeout and fallback chain.
- LLM circuit breaker.
- Startup readiness checks.
- Audit logging.
- Privacy consent/export/delete API surface.

Resilience Still Needed

- Hardware disconnect validation on actual RV1126.

- Road testing under real sensor noise.
- Production Redis instead of development fallback.
- Shared circuit breaker state for multi-instance backend.
- Real TLS certificate pins.
- Full crash reporting and alerting.
- Load tests.
- Backup/restore tests.
- Compatibility matrix for OBD dongles, watches, phones, and vehicles.

11. How AI Works Inside thun.ai

Use this if the founder asks: “If we speak about AI, how does it work within thun.ai?”

The Clean Explanation

AI in thun.ai has three roles. First, deterministic sensor intelligence calculates the driver's stress state from OBD, biometrics, and vision signals. Second, a small/on-device language model can turn a detected driving event into a calm one-sentence intervention. Third, cloud LLMs generate post-drive coaching, therapist responses, and practice scenarios. We deliberately do not let generative AI directly control the car or make safety-critical control decisions.

Real-Time AI Path

Real-time drive support is intentionally conservative:

1. OBD, biometrics, and CV signals arrive.
2. Stress score is computed deterministically.
3. Rule logic checks threshold, speed gate, stall condition, emergency override, and cooldown.
4. If an intervention is allowed, the SLM or predefined prompt produces calm instruction text.
5. Voice/HUD deliver the instruction.

This means:

- The AI does not steer.
- The AI does not brake.
- The AI does not replace the driver.
- The AI helps the driver stay regulated.

Post-Drive AI Path

Post-drive AI is where larger models are useful:

1. The drive summary is packaged.
2. LLM receives route, average stress, peak stress, stress events, and corridor outcomes.
3. LLM writes a warm confidence narrative.
4. High-stress events can generate synthetic practice scenarios.
5. Backend stores confidence trajectory so progress can be tracked.

Therapist AI Path

Therapist AI is a parked-only coaching feature:

1. User requests therapist.
2. System checks stationary/RPM condition.
3. Conversation is trimmed and sanitized.
4. LLM responds in short, non-clinical, calming language.
5. TTS speaks it when safe.

VLM/Perception Path

Vision can operate in two ways:

- Classical object detection/perception models for real-time signals.
- Optional VLM scene analysis for richer context outside the hard real-time loop.

Do not overclaim:

The current code has a perception backend and edge CV scaffolding, but the real production CV models still need to be integrated and validated.

12. Test Status As Of This Workspace Review

Commands attempted on April 21, 2026 in this local workspace:

Backend

Command:

npm.cmd --workspace backend test -- --runInBand

Result:

- 6 test suites passed.
- 115 tests passed.
- 0 backend test failures.

Note:

- Running backend Jest in parallel hit a local Windows spawn EPERM, so serial --runInBand was used.

Mobile

Command:

npm.cmd --workspace mobile test -- --runInBand

Result:

- 4 test suites passed.
- 1 test suite failed.
- 62 tests passed.
- 1 test failed.

Failing test:

- mobile/__tests__/OBDService.test.js
- Failure is a Jest mock/timer expectation around setTimeout, not a demonstrated product-flow failure.

Python

Command attempted:

py -3 -m pytest tests -q

Result:

- Could not run because pytest is not installed in the available Python environment.
- python command is not on PATH; only Windows py.exe is available.

Edge

Result:

- Edge C++ tests exist in edge/test.
- No local edge build artifacts were present.
- Edge tests were not executed in this session.
- CMake test setup uses GoogleTest FetchContent, which may require network/dependency setup.

Founder wording:

Backend is clean on tests. Mobile is nearly clean with one harness-level OBD timer test failing. Python and edge tests need environment setup before I can claim current pass counts.

13. What To Demo Live

Demo Path

1. Onboarding
2. Trigger profile and vehicle width
3. Pre-drive route comparison
4. Confidence corridor preview
5. Drive screen with stress gauge
6. Simulated stress intervention
7. Stall protocol
8. Therapist stationary gate
9. Post-drive report
10. Settings, privacy, sync/runtime health

What To Say During Demo

Onboarding:

We start by building a driver-state profile, not just an account. This tells the system what situations are likely to create overload for this person.

Route scoring:

We are not ranking routes only by ETA. We rank them by calmness and exposure to triggers.

Confidence corridor:

This is a key product differentiator. In tight spaces, the driver does not need a vague warning. They need a measurable answer: do I fit, how much space do I have, and what speed should I maintain?

Drive screen:

The drive loop runs locally. Sensor inputs become a stress score, and interventions only happen when thresholds are crossed.

Speed gate:

The system also knows when to stay quiet. Above the speed gate, voice prompts are suppressed to avoid adding distraction.

Stall:

Many anxious drivers are not dangerous because they drive aggressively. They become unsafe because they freeze. This is why stall recovery is part of the product.

Post-drive:

The drive should not end as a scary memory. It should become a structured confidence-building moment.

Therapist:

This is parked-only support. It is not a medical product and it is not active while driving.

14. Founder Questions And Strong Answers

Is this ADAS?

No. ADAS is mostly vehicle/environment-centric. thun.ai is driver-state-centric. We can consume ADAS-like signals, but the product goal is different: reduce human overload and build confidence.

Is this safe?

Safe in design philosophy, but not yet production-safety-certified. The real-time loop is deterministic, speed-gated, and local. Generative AI does not control the vehicle. But safety certification and real hardware validation are still future work.

What is the moat?

The moat is the full loop: profile → calm route → stress detection → intervention → stall recovery → post-drive confidence → confidence memory. A single feature can be copied. The longitudinal driver-state loop is harder to copy.

Why would OEMs care?

Because it gives them a practical, human-centered safety story without promising autonomy. It is relevant for first-time drivers, family cars, dense urban markets, and customers who avoid driving because it feels overwhelming.

What can we pilot first?

Mobile plus OBD plus watch plus backend is the fastest pilot wedge. Edge RV1126 can run in parallel as the hardware validation track.

What is the biggest risk?

Hardware validation and real perception quality. The product experience is visible, but the production-grade sensor truth needs field proof.

What happens if the cloud fails?

The real-time safety loop continues locally. Post-drive feedback, therapist, and sync are cloud-dependent but can fall back, queue, or return deterministic responses.

What happens if OBD disconnects?

Mobile OBD reconnects with backoff. Stress scoring can reweight available inputs. The user should see degraded mode. Real-world dongle compatibility still needs testing.

What happens if the LLM fails?

Backend tries Gemini, then Claude, then OpenAI. If all fail or circuits are open, it returns a deterministic fallback narrative.

How soon can we test hardware?

If the unit, models, BLE key, camera, and CAN are ready, we can smoke-test the same day. A credible controlled road validation takes 1 to 2 weeks.

15. What Not To Say

Avoid:

- “Self-driving”
- “Autonomous intervention”
- “Certified safety system”
- “Production-ready for OEM”
- “CV accuracy is solved”
- “Therapist is a medical product”
- “Hardware is validated” unless it has actually passed target-device tests

Use instead:

- “Driver-state-aware safety layer”
- “Confidence-aware driving support”
- “Controlled pilot-ready software path”
- “Hardware validation track”
- “Human-centered safety”
- “Generative AI is used for coaching and wording, not vehicle control”

16. Near-Term Action Plan

Before The Founder Call

1. Keep one clean demo flow ready.
2. Have a short architecture diagram open from docs/ARCHITECTURE.md.
3. Have this briefing open.
4. Be ready to say exact test status.
5. Be ready to explain the hardware gap without sounding defensive.
6. Decide the ask: funding, pilot approval, hardware procurement, or roadmap agreement.

Next Engineering Milestones

1. Fix the mobile OBDService timer test.
2. Set up Python test environment and run `pytest`.
3. Set up edge build environment and run C++ tests.
4. Proxy Google Maps and TTS through backend for production.
5. Replace confidence corridor route simulation with real side-clearance signals.
6. Integrate and validate RKNN CV models.
7. Run RV1126 self-test on real hardware.
8. Run static vehicle and low-speed controlled drive tests.
9. Complete consent UX and deletion job workflow.
10. Add monitoring, alerting, load tests, and CI/CD migrations.

17. Closing Statement

End with this:

The strongest thing about thun.ai is that it does not try to win by promising the car will drive itself. It wins by helping the driver remain calm, capable, and informed in moments where stress usually creates risk. The product loop is already visible in software. The next proof point is controlled hardware and real-road validation.

Prepared for a 60 to 75 minute founder call about what is built, what is not built, how the system is configured, how data moves through thun.ai, how AI works inside the product, and how strong the product is today.